Centro Universitário de Excelência
Sistemas de Informação

Introdução ao Pré-processamento de Dados

Autores: Andrey Mota de Oliveira
Claudio dos Santos Junior
Gabriel da Silva e Silva
Louise de Souza Santino

Feira de Santana, 2026

Agenda

O objetivo dessa apresentação é demonstrar o desenvolvimento de uma biblioteca de pré-processamento de dados em Python e sua aplicação no **dataset Spotify Songs for ML & Analysis.**

1. Contextualização:
Objetivo, problema e aplicação do que foi feito baseado no dataset.

2. Classificação das Operações e seus tipos:
Classificação das operações, destacando suas características.

3. Classes & Métodos:
Apresentação das classes e métodos.

4. Restrições:
Restrições do dataset.

5. Métodos:
Aplicação dos métodos no dataset e seus exemplos.

6. Pré-processamento & Resultados:
Principais resultados obtidos e considerações sobre o pré-processamento.

7. Considerações Finais e Conclusão:
Considerações sobre a limitação do pré-processamemento e conclusão final

Contextualização

● A Dendê Softhouse é uma empresa focada em soluções digitais e análise de dados, responsável pelo aplicativo Dendê Eventos.

● Na primeira etapa, foi desenvolvida uma biblioteca de análise exploratória de dados, permitindo identificar problemas no dataset.

● Foram encontrados desafios como dados ausentes, escalas diferentes e variáveis categóricas não tratadas.

● Nesta etapa, foi desenvolvida uma biblioteca Python de pré-processamento com classes como **MissingValueProcessor**, **Scaler**, **Encoder** e **Preprocessing**.

● As técnicas foram implementadas com recursos nativos e aplicadas ao dataset **Spotify Songs for ML & Analysis**.

Classificação dos tipos de operação

Tratamento de Dados Ausentes	MissingValueProcessor
Transformação de Escala	Scaler
Codificação de Dados Categóricos	Encoder

Classes & Métodos Usados

MissingValueProcessor	Scaler	Encoder	Preprocessing
__init__ _get_target_columns isna notna fillna dropna	__init__ _get_target_columns minMax_scaler standard_scaler	__init__ label_encode def oneHot_encode	__init__ _validate_dataset_shape scale encode isna notna fillna dropna

Restrições do Dataset

1. Recursos nativos em linguagem de programação Python
2. Sem biblioteca externas (numpy, pandas)
3. Técnicas manuais implementadas do zero
4. Dicionário de Listas
5. Dataset Spotify Songs for ML & Analysis
6. Usar métodos da biblioteca

Dados Ausentes ou Nulos

isna(self, columns: Set[str] = None)
Exemplo: *Ver problema*

def notna(self, columns: Set[str] = None) -> Dict[str, List[Any]]:
Exemplo: *Pegar dados válidos*

def fillna(self, columns: Set[str] = None, value: Any = 0) -> Dict[str, List[Any]]:
Exemplo: *Corrigir*

def dropna(self, columns: Set[str] = None) -> Dict[str, List[Any]]:
Exemplo: *Apagar*

Transformadores de Escala

def minMax_scaler •	• def standard_scaler
Escala fixa (0 a 1)	Média 0, desvio 1
Sensivel a outliers	Lida melhor com outliers
Mantém proporção	Centraliza os dados

≠

$$x' = \frac{x - min}{max - min}$$

$$z = \frac{x - \mu}{\sigma}$$

Transformação de Escala

Codificadores de valores categóricos

label_encode(self, columns: Set[str])
Exemplo: *Quando tem ordem*

def oneHot_encode(self, columns: Set[str]) -> Dict[str, List[Any]]:
Exemplo: *Quando não tem ordem*

Codificadores de valores categóricos

- **Estudo de caso**

  Um cientista de dados está analisando o dataset track_data_final.csv e precisa codificar duas variáveis categóricas para um modelo de Machine Learning:

Variável	Valores
Explicit	True, False
Album_type	‘álbum’, ‘single’, ‘compilation’

<u>Qual das seguintes abordagens é a mais adequada para codificar essas variáveis?</u>

A) Aplicar Label Encoder em ambas as variáveis.
B) Aplicar Label Encoder em explicit e One-Hot Encoder em album_type.
C) Aplicar One-Hot Encoder em explicit e Label Encoder em album_type.

Pré-processamento do dataset Spotify Songs

```
============================================================
ANÁLISE DE PRÉ-PROCESSAMENTO - SPOTIFY
============================================================
Dataset carregado: 8778 músicas

1. DATASET ORIGINAL: 8778 linhas


2. VALORES NULOS
   Colunas numéricas:
     - artist_popularity: 4 nulos (0.0%)
     - artist_followers: 4 nulos (0.0%)
   Colunas categóricas:

3. TRATAMENTO DE NULOS
   - fillna(value=0): todos os nulos numéricos preenchidos


4. ESCALONAMENTO MIN-MAX
   ANTES:
     track_popularity: min=0.00, max=100.00
     artist_popularity: min=0.00, max=100.00
   DEPOIS (0 a 1):
     track_popularity: min=0.00, max=1.00
     artist_popularity: min=0.00, max=1.00
```

```
5. LABEL ENCODING
   ANTES:
     explicit: 2 categorias
     album_type: 3 categorias
   DEPOIS (valores numéricos):
     explicit: [0, 0, 0, 1, 0]
     album_type: [1, 2, 0, 0, 0]


6. ONE-HOT ENCODING
   Original:
     album_type: ['album', 'single', 'album', 'compilation', 'single']
     explicit: ['TRUE', 'FALSE', 'TRUE', 'FALSE', 'TRUE']
   Após One-Hot (novas colunas):
     album_type_album: [1, 0, 1, 0, 0]
     album_type_compilation: [0, 0, 0, 1, 0]
     album_type_single: [0, 1, 0, 0, 1]
```

Resultados

- Todos os testes passaram

```
----------------------------------------------------------------------
Ran 12 tests in 0.008s

OK
```

Considerações finais

- Uso exclusivo de recursos nativos da linguagem de programação Python limitou o desempenho em grandes volumes de dados.

- Implementação manual exigiu maior cuidado para evitar distorções nas transformações.

- Biblioteca aplicada com sucesso no dataset **Spotify**, tratando valores ausentes, escalas e variáveis categóricas.

- A experiência com implementações manuais reforçou conceitos fundamentais da área.

- É possível construir soluções robustas com Python puro, desde que haja planejamento adequado.

Referências

- BARBETTA, Pedro Alberto; REIS, Marcelo Menezes; BORNIA, Antonio Cezar.**Estatística para Cursos de Engenharia e Informática**. 4. ed. São Paulo: Atlas, 2014.

- KAGGLE. Spotify Songs for ML & Analysis (8700+ tracks). **Dataset de AlyAhmedTS13.** Disponível em: https://www.kaggle.com/datasets/alyahmedts13/spotify-songs-for-ml-and-analysis over-8700-tracks. Acesso em: 15 mar. 2026

- PEDREGOSA, F., VAROQUAUX, G. AND GRAMFORT, A. (2011) "Scikit-learn: Machine Learning in Python", **Journal of Machine Learning Research**, Disponível em: https://jmlr.csail.mit.edu/papers/v12/pedregosa11a.html, Acesso em: 15 mar. 2026.

- WILSON, G. ET AL. (2014) "Best Practices for Scientific Computing**", PLOS Biology,** Disponível em: https://journals.plos.org/plosbiology/article?id=10.1371/journal.pbio.1001745, Acesso em: 18 mar. 2026.